

Trabalho - N2 - API REST + ORM + DTO

Data de entrega: 07/05 até 23h59.

Apresentação: 08/05.

Objetivo: Evoluir a ApiClinica para usar ORM e DTO.

Equipe: Pode ser realizado em grupo de até cinco pessoas.

Entrega:

- Código-fonte funcional. Projeto completo no formato .zip (evitar .rar).
- Apenas uma pessoa do grupo deve fazer a entrega no Teams.
- Entregas atrasadas NÃO serão aceitas.

Avaliação:

- Código = 7 pontos (70%).
- Apresentação = 3 pontos (30%).

Sobre o código

- O código deve estar funcional e será testado no Postman.
- A API deve ser implementada em C#, .NET 10.
- A API deve conter:
 - Diretório Models com Paciente, Médico e Consulta.
 - Diretório Controllers com PacientesController, MedicosController e ConsultasController.
 - Diretório Data com ApplicationDbContext.
 - Diretório DTOs com:
 - PacienteCreatedDTO, PacienteReadDTO, PacienteUpdateDTO
 - MedicoCreatedDTO, MedicoReadDTO, MedicoUpdateDTO
 - ConsultaCreatedDTO, ConsultaReadDTO, ConsultaUpdateDTO

- Diretório Mappers com PacienteMapper, MedicoMapper e ConsultaMapper implementados de forma manual.
- Controllers devem receber e retornar apenas DTOs.
- PacientesController deve conter os métodos GET, GET by Id, POST, PATCH, e DELETE.
- PacientesController deve validar as seguintes informações nos métodos PATCH e POST:
 - Email com formato válido.
 - Telefone com formato válido.
 - Data de nascimento não pode ser no futuro.
 - CPF válido (CPF não pode ser alterado no PATCH).
 - Ao criar Paciente (POST) deve validar se já existe um Paciente com o mesmo CPF.
- MedicosController deve conter os métodos GET, GET by Id, POST, PATCH, e DELETE.
- MedicosController deve validar as seguintes informações nos métodos PATCH e POST:
 - Email com formato válido.
 - Telefone com formato válido.
- ConsultasController deve conter os métodos GET, GET by Id, POST, PATCH, e DELETE.
- ConsultasController deve validar as seguintes informações nos métodos PATCH e POST:
 - Id do paciente é válido, paciente existe.
 - Id do médico é válido, médico existe.
 - Não deve ser possível agendar consulta no passado.
 - Não deve ser possível agendar consulta em dois horários iguais para o mesmo médico.
 - Não deve ser possível agendar consulta em dois horários iguais para o mesmo paciente.
 - Não pode haver sobreposição de horários.
 - Cada consulta tem duração de 30 minutos.
 - Exemplo: 10:00 - 10:30, 10:15 - 10:45 (sobreposição)
 - Se o id do paciente ou do médico for alterado, essa regra deverá ser validada novamente.
- No método PATCH, se o sistema receber *null* para algum campo, deve-se considerar que esse campo não deve ser atualizado.

- Não deve ser possível remover (DELETE) um Paciente ou um Médico que tenha Consultas no futuro.
- Atenção: todos os métodos (GET, POST, PATCH, DELETE) dos pacientes, médicos e consultas devem fazer uso do DbContext.
- O banco de dados utilizado deve ser o SQLite.
- Estrutura do Paciente, Médico e Consulta:

```
Paciente {  
    Id,  
    Nome,  
    Email,  
    Telefone,  
    DataNasc,  
    Cpf  
}
```

```
Médico {  
    Id,  
    Nome,  
    Email,  
    Telefone,  
    CRM  
}
```

```
Consulta {
```

```
Id,  
PacientId,  
MedicId,  
DataHora  
}
```

Sobre a apresentação

- A apresentação será feita somente para a professora.
- Serão feitas perguntas para os membros do grupo, de forma aleatória. Portanto, todos os membros da equipe devem ter conhecimento total do código desenvolvido.

Critérios de avaliação da apresentação:

- Domínio do código desenvolvido, considerando todos os membros do grupo.
- Domínio ao responder as perguntas.
- Código funcional com testes no Postman.